

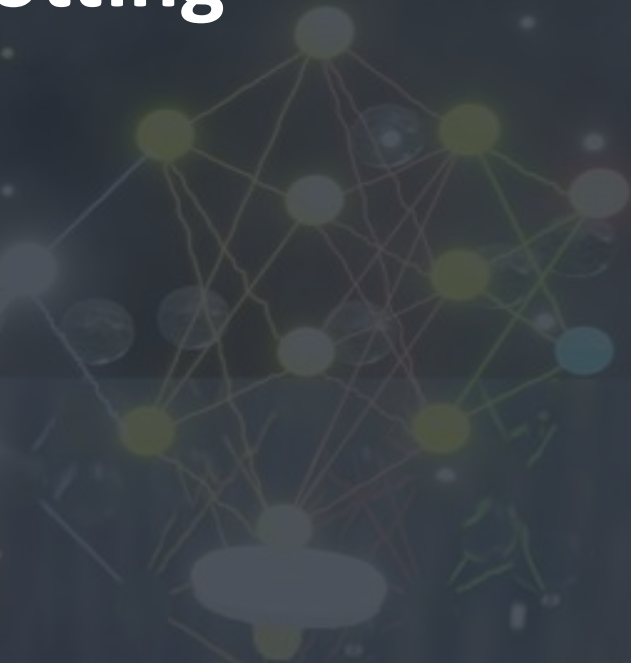
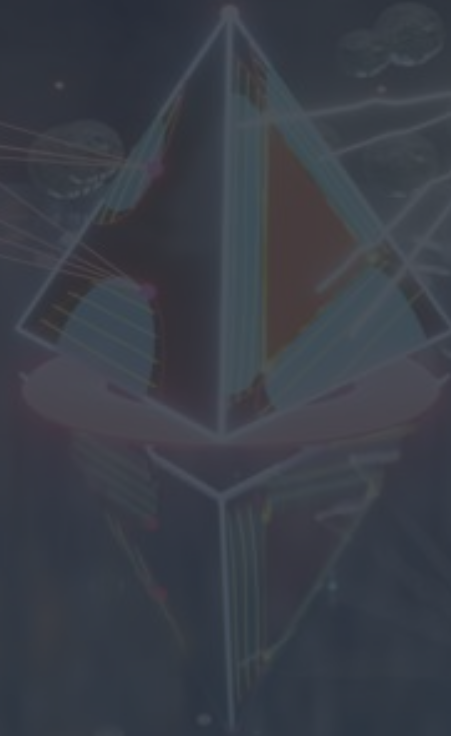
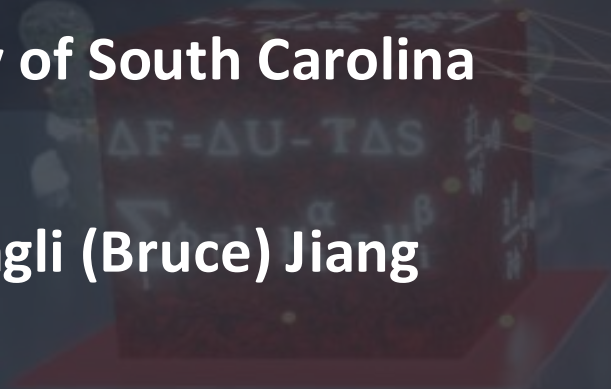
Python Foundation II

Dictionaries, Functions, Data, and Plotting

Fall 2026

University of South Carolina

Prof. Shengli (Bruce) Jiang



Python data structures

List: Ordered, changeable, and allows mixed data types

```
sample = ["run_01", 350, 2.0, True]

sample[1] = 360
sample.append("complete")

print(sample)

['run_01', 360, 2.0, True, 'complete']
```

Tuple: Ordered, fixed, and allows mixed data types

```
experiment = ("run_01", 350, 2.0, True)

name, temperature, pressure, complete = experiment

print(f"name: {name}")
print(f"temperature: {temperature}")
print(f"pressure: {pressure}")
print(f"complete: {complete}")

name: run_01
temperature: 350
pressure: 2.0
complete: True
```

Python data structures

NumPy Array: Ordered, changeable, usually one data type, and designed for numerical computation

```
import numpy as np

temperature = np.array([300, 350, 400])

temperature[0] = 310
temperature = temperature + 10

print(temperature)

[320 360 410]
```

```
temperature = np.array([300, 350, 400])

print(temperature.dtype)

temperature[0] = 'hot'

int64
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[16], line 5
      1 temperature = np.array([300, 350, 400])
      3 print(temperature.dtype)
----> 5 temperature[0] = 'hot'

ValueError: invalid literal for int() with base 10: 'hot'
```

```
temperature = np.array([300, 350, 400])

print(temperature.dtype)

temperature[0] = '110'

print(temperature)

int64
[110 350 400]
```

Python data structures

NumPy Array: Ordered, changeable, usually one data type, and designed for numerical computation

```
import numpy as np

temperature = np.array([300, 350, 400])

temperature[0] = 310
temperature = temperature + 10

print(temperature)

[320 360 410]
```

```
temperature = np.array(['hot', 350, 400])

print(temperature.dtype)

print(temperature + 1)
```

<U21

UFuncTypeError

Traceback (most recent

call last)

Cell In[22], line 5

```
1 temperature = np.array(['hot', 350, 400])
3 print(temperature.dtype)
----> 5 print(temperature + 1)
```

UFuncTypeError: ufunc 'add' did not contain a loop with signature matching types (dtype('<U21'), dtype('int64')) -> None

Python data structures

Dictionary: Labeled, changeable, and allows mixed value types

```
experiment = {  
    "name": "run_01", # key: value  
    "temperature": 350,  
    "pressure": [2.0, 3.0, 4.0],  
    "complete": True  
}  
  
experiment["temperature"] = 360 # change value  
experiment["operator"] = "Alex" # add value  
experiment["pressure"].append(5.0)  
  
print(experiment["temperature"])  
print(experiment["operator"])  
print(experiment["pressure"])
```

360

Alex

[2.0, 3.0, 4.0, 5.0]

Python data structures

Dictionary: Labeled, changeable, and allows mixed value types

```
experiment = {  
    "name": "run_01", # key: value  
    "temperature": 350,  
    "pressure": [2.0, 3.0, 4.0],  
    "complete": True  
}  
  
print(experiment.keys())      # keys  
print(experiment.values())    # values  
print('name' in experiment)   # if a key exists in the dict  
print(len(experiment))        # length of the dict
```

```
dict_keys(['name', 'temperature', 'pressure', 'complete'])
```

```
dict_values(['run_01', 350, [2.0, 3.0, 4.0], True])
```

```
True
```

```
4
```

Python data structures: summary

Type	Use it when	Defining features	Example
List	You need an ordered collection whose contents may change	Ordered, mutable, allows mixed types	<code>temperatures = [300, 350, 400]</code>
Tuple	You need a fixed record that should not change	Ordered, immutable, supports unpacking	<code>run = ("R01", 350, 2.0)</code>
Dictionary	Each value should have a meaningful label	Key-value mapping, mutable, unique keys	<code>run = {"T": 350, "P": 2.0}</code>
NumPy Array	You need numerical calculations on many values	Multidimensional, mutable, one dtype, vectorized operations	<code>T = np.array([300, 350, 400])</code>

A *function* packages reusable instructions

```
def celsius_to_kelvin(T_c):  
    T_k = T_c + 273.15  
    return T_k  
  
T_c = 25  
T_k = celsius_to_kelvin(T_c)  
print(f"{T_c:.1f} C is {T_k:.2f} K")  
  
T_c = 100  
T_k = celsius_to_kelvin(T_c)  
print(f"{T_c:.1f} C is {T_k:.2f} K")
```

25.0 C is 298.15 K
100.0 C is 373.15 K

Without **return**, the calculation happens inside the function, but the result is not available outside the function.

```
def celsius_to_kelvin(T_c):  
    T_k = T_c + 273.15  
  
result = celsius_to_kelvin(25)  
print(f"Returned value is {result}")
```

Returned value is None

```
def celsius_to_kelvin(T_c):  
    T_k = T_c + 273.15  
    return  
  
result = celsius_to_kelvin(25)  
print(f"Returned value is {result}")
```

Returned value is None

Bad habits make simple functions difficult to read

```
def abc(a, s2, d):  
    z = a + s2  
    z = z + d  
    x = z * 2  
    return x
```

Don't write functions with useless variable names that make it impossible for other people and your future self to understand your code!



```
def twice_the_sum(x1, x2, x3):  
    total = x1 + x2 + x3  
    return 2 * total
```

```
def reactor_status(temperature, pressure):  
    """Classify reactor operating conditions.  
  
    temperature is measured in K.  
    pressure is measured in bar.  
    Returns normal, warning, or shutdown.  
    """  
  
    if temperature > 500 or pressure > 10:  
        return "shutdown"  
    elif temperature > 450 or pressure > 8:  
        return "warning"  
    else:  
        return "normal"
```

Always add docstrings for complex functions!

Writing functions in Python: multiple outputs

```
def summarize_temperature(temperatures):  
    minimum = min(temperatures)  
    maximum = max(temperatures)  
    average = sum(temperatures) / len(temperatures)
```

```
    return minimum, maximum, average
```

```
result = summarize_temperature([300, 350, 400])
```

```
print(result)  
print(type(result))
```

```
(300, 400, 350.0)  
<class 'tuple'>
```

```
def summarize_temperature(temperatures):  
    T_min = min(temperatures)  
    T_max = max(temperatures)  
    T_average = sum(temperatures) / len(temperatures)
```

```
    return {  
        "minimum": T_min,  
        "maximum": T_max,  
        "average": T_average  
    }
```

```
summary = summarize_temperature([300, 350, 400])
```

```
print(summary["minimum"])  
print(summary["average"])
```

```
300  
350.0
```

Python returns one object. Use a tuple for a small fixed group of results. Use a dictionary when named results make the code clearer.

Writing loops in Python

```
total = 0

for number in range(1, 4):
    total = total + number
    print(f"number = {number}, total = {total}")
```

```
number = 1, total = 1
number = 2, total = 3
number = 3, total = 6
```

The loop variable takes one value at a time, and the indented code runs once for each value.

range(4) gives 0, 1, 2, 3.

```
names = ["Run 1", "Run 2", "Run 3"]
temperatures = [300, 350, 400]
```

```
for name, temperature in zip(names, temperatures):
    print(f"{name} has temperature {temperature} K")
```

```
Run 1 has temperature 300 K
Run 2 has temperature 350 K
Run 3 has temperature 400 K
```

Loop through two sequences together.

zip() stops when the shortest sequence ends.

```
runs = ["Run 1", "Run 2", "Run 3", "Run 4"]
temperatures = [300, 350, 400]
pressures = [1.0, 2.0]
```

```
print(list(zip(runs, temperatures, pressures)))

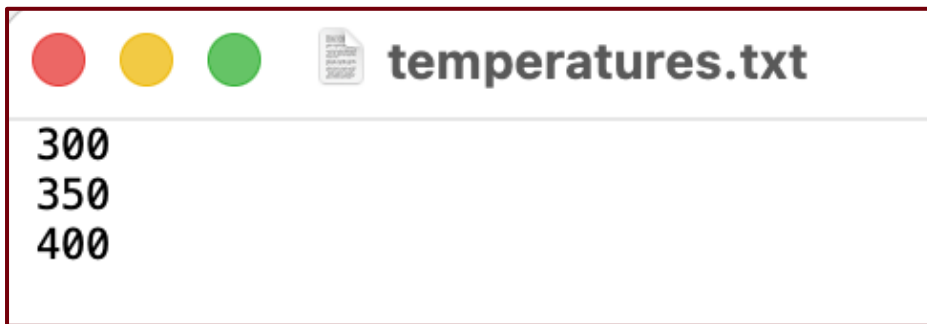
[('Run 1', 300, 1.0), ('Run 2', 350, 2.0)]
```

I/O in Python

Write a file

```
temperatures = [300, 350, 400]

with open(
    "temperatures.txt",
    "w",
    encoding="utf-8"
) as file:
    for temperature in temperatures:
        file.write(f"{temperature}\n")
```



Read a file

```
with open(
    "temperatures.txt",
    "r",
    encoding="utf-8"
) as file:
    content = file.read()

print(content)
```

300
350
400

Mode	Meaning
"r"	Read an existing file
"w"	Create or overwrite a file
"a"	Add content to the end

I/O using Pandas

Pandas is frequently used for machine learning model training, especially when the data are stored as a **table**.

- Read CSV, Excel, SQL, JSON
- Inspect data types
- Rename features
- Handle missing values

```
import pandas as pd

data = {
    'temperature': [320, 340, 360, 380, 400, 420],
    'time':        [1.0, 1.2, 1.4, 1.6, 1.8, 2.0],
    'catalyst':     ['A', 'A', 'B', 'B', 'C', 'C'],
    'conversion':  [0.42, 0.55, 0.67, 0.74, 0.81, 0.86],
    'yield':       [0.31, 0.44, 0.56, 0.62, 0.65, 0.66]
}

df = pd.DataFrame(data)

df
```

[54]:

	temperature	time	catalyst	conversion	yield
0	320	1.0	A	0.42	0.31
1	340	1.2	A	0.55	0.44
2	360	1.4	B	0.67	0.56
3	380	1.6	B	0.74	0.62
4	400	1.8	C	0.81	0.65
5	420	2.0	C	0.86	0.66

Pandas Examples

```
import pandas as pd
```

```
df = pd.read_csv("manufacturing_data_with_issues.csv")
```

```
df.head()
```

`df.head()`: What do the first rows look like?

	Batch ID	Process Date	Reactor Temp C	Pressure bar	Flow Rate L min	Catalyst Type	Defect Count	Conversion Percent	Quality Notes
0	B001	2026-08-01	310	5.2	12.5	A	0.0	94.2	normal run
1	B002	2026-08-02	315	5.4	12.8	A	1.0	92.5	normal run
2	B003	2026-08-03	NaN	5.3	12.7	B	2.0	89.1	missing temperature
3	B004	2026-08-04	999	5.5	13.0	B	8.0	45.0	temperature sensor error
4	B005	2026-08-05	325	NaN	13.1	A	1.0	91.8	missing pressure

`df.shape`: How many rows and columns?

```
df.shape
```

(22, 9)

`df.columns`: What are the labels?

```
df.columns
```

```
Index(['Batch ID', 'Process Date', 'Reactor Temp C', 'Pressure bar',  
      'Flow Rate L min', 'Catalyst Type', 'Defect Count',  
      'Conversion Percent', 'Quality Notes'],  
      dtype='object')
```

Pandas Examples

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 22 entries, 0 to 21
Data columns (total 9 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   Batch ID              22 non-null    object
 1   Process Date          22 non-null    object
 2   Reactor Temp C        21 non-null    object
 3   Pressure bar          21 non-null    float64
 4   Flow Rate L min       21 non-null    float64
 5   Catalyst Type         21 non-null    object
 6   Defect Count          21 non-null    float64
 7   Conversion Percent    21 non-null    float64
 8   Quality Notes         22 non-null    object
dtypes: float64(4), object(5)
memory usage: 1.7+ KB
```

`df.info()`: Data information summary.

```
df.describe()
```

	Pressure bar	Flow Rate L min	Defect Count	Conversion Percent
count	21.000000	21.000000	21.000000	21.000000
mean	7.561905	13.761905	2.904762	81.590476
std	4.106395	3.845709	2.447545	15.556700
min	5.200000	-2.000000	0.000000	39.800000
25%	5.900000	13.100000	1.000000	78.800000
50%	6.800000	14.600000	2.000000	85.900000
75%	7.600000	15.500000	4.000000	90.400000
max	25.000000	16.800000	9.000000	101.500000

`df.describe()`: Summarizes numerical columns by default. It recognizes Nan, but excludes missing values from the calculation.

Handling missing data

Step 1: Convert invalid text to missing data

```
df["Reactor Temp C"] = pd.to_numeric(
    df["Reactor Temp C"],
    errors="coerce"
)
```

Text like “not recorded” becomes NaN.

Step 2: Find missing values

```
df.isna().sum()
```

Batch ID	0
Process Date	0
Reactor Temp C	2
Pressure bar	1
Flow Rate L min	1
Catalyst Type	1
Defect Count	1
Conversion Percent	1
Quality Notes	0
dtype: int64	

`isna()` identifies missing values such as NaN, None, and NaT.

Handling missing data

Step 3: Handle missing values

```
df_clean = df.dropna(  
    subset=["Conversion Percent"]  
).copy()
```

Remove rows with a missing prediction target

```
numeric_features = [  
    "Reactor Temp C",  
    "Pressure bar",  
    "Flow Rate L min",  
    "Defect Count"  
]  
  
for column in numeric_features:  
    median = df_clean[column].median()  
    df_clean[column] = df_clean[column].fillna(median)
```

Fill missing numerical
features with their medians

```
df_clean["Catalyst Type"] = (  
    df_clean["Catalyst Type"]  
    .fillna("Unknown")  
)
```

Fill a missing categorical feature

```
df_clean.isna().sum()
```

Batch ID	0
Process Date	0
Reactor Temp C	0
Pressure bar	0
Flow Rate L min	0
Catalyst Type	0
Defect Count	0
Conversion Percent	0
Quality Notes	0
dtype: int64	

Pandas I/O and converting to NumPy

```
df_clean.to_csv(  
    "manufacturing_data_clean.csv",  
    index=False  
)
```

index=False prevents pandas from adding the row index as an extra CSV column

Optional Excel example

```
sheets = pd.read_excel(  
    "manufacturing_data.xlsx",  
    sheet_name=["Reactor Data", "Quality Data"]  
)  
  
reactor_df = sheets["Reactor Data"]  
quality_df = sheets["Quality Data"]
```

```
feature_columns = [  
    "Reactor Temp C",  
    "Pressure bar",  
    "Flow Rate L min",  
    "Defect Count"  
]  
  
X = df_clean[feature_columns].to_numpy(dtype=float)  
  
y = df_clean["Conversion Percent"].to_numpy(dtype=float)  
  
print(X.shape)  
print(X.dtype)  
print(X[:3])  
  
print(y.shape)  
print(y[:3])  
  
(21, 4)  
float64  
[[310.    5.2  12.5   0. ]  
 [315.    5.4  12.8   1. ]  
 [370.    5.3  12.7   2. ]]  
(21,)  
[94.2 92.5 89.1]
```

I highly recommend using NumPy rather than pandas for mathematical operations!

Python plotting

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(4, 4))

ax.scatter(
    df_clean["Reactor Temp C"],
    df_clean["Defect Count"]
)

ax.set_xlabel("Reactor temperature (C)")
ax.set_ylabel("Defect count")
ax.set_title("Defects versus reactor temperature")
ax.grid(alpha=0.3)

fig.tight_layout()

fig.savefig(
    "defects_vs_temperature.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```

Figure

The complete canvas

Axes

The plotting area
with x-axis, y-axis,
data,
labels, and
annotations



Prefer the object-oriented style using **ax**.

Plotting exposes values that still need validation

```
fig, ax = plt.subplots(figsize=(5, 4))

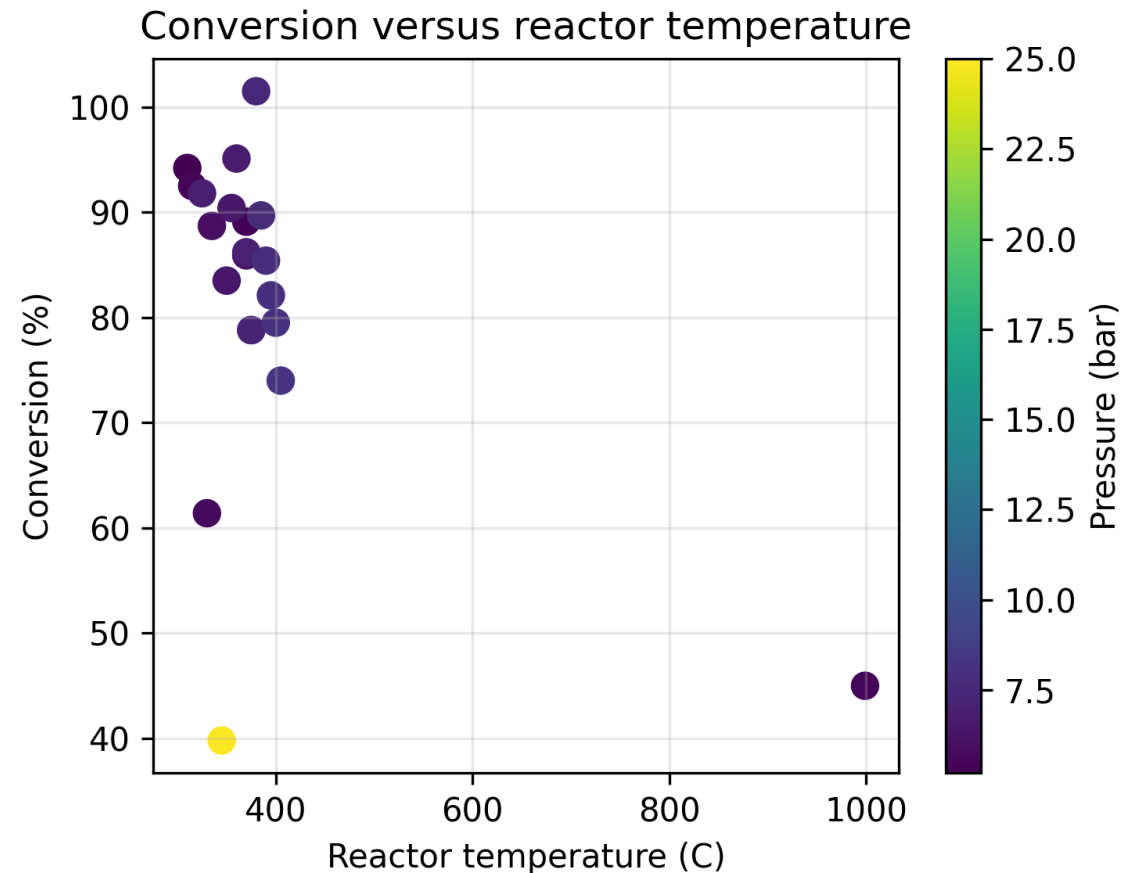
points = ax.scatter(
    df_clean["Reactor Temp C"],
    df_clean["Conversion Percent"],
    c=df_clean["Pressure bar"],
    cmap="viridis",
    s=60
)

ax.set_xlabel("Reactor temperature (C)")
ax.set_ylabel("Conversion (%)")
ax.set_title("Conversion versus reactor temperature")
ax.grid(alpha=0.3)

colorbar = fig.colorbar(points, ax=ax)
colorbar.set_label("Pressure (bar)")

fig.tight_layout()
fig.savefig(
    "conversion_temperature_pressure.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```



Color has three different jobs

Identity

Which group?



qualitative

Magnitude

How much?



sequential

Deviation

Above or below a reference?



diverging

Decide the job before choosing the color map.

Match the color map to the data type

Qualitative

Catalyst A, B, C



Sequential

Concentration from low to high



Diverging

Residual around zero



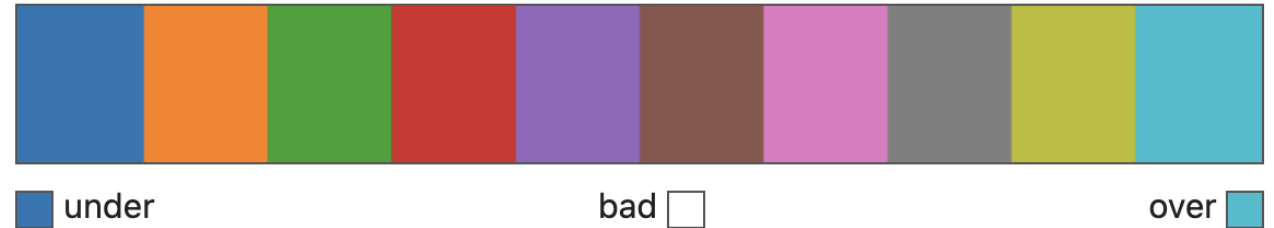
Match the color map to the data type

Qualitative

Catalyst A, B, C

```
plt.get_cmap('tab10') # categories
```

tab10



Sequential

Concentration from low to high

```
plt.get_cmap('viridis') # ordered values
```

viridis

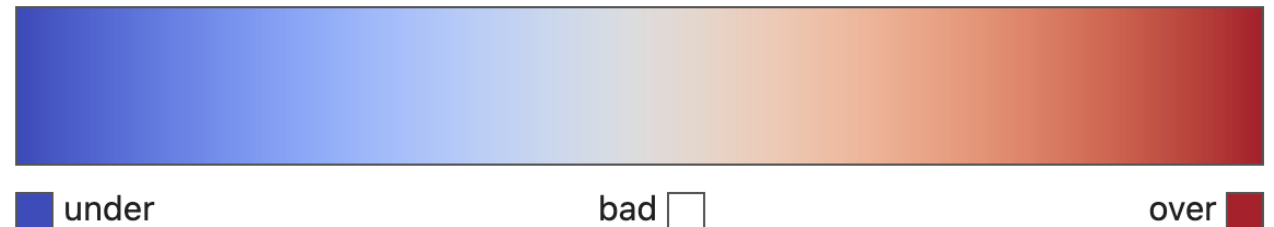


Diverging

Residual around zero

```
plt.get_cmap('coolwarm') # centered deviations
```

coolwarm



Perceptual color should preserve the data order

A scientific color map should change smoothly in perceived lightness.

Better for ordered values



viridis

Risky for ordered values



rainbow-like

Check the plot in grayscale when order matters!

rainbow scale



rainbow converted to grayscale



Accessible plots do not rely on color alone

Repeat important distinctions through a second visual channel.

Color

blue, teal, gold

Marker

circle, square, triangle

Line style

solid, dashed, dotted

Direct label

write the group name near the data



Color should help, not carry the entire explanation.

A small palette is easier to reuse consistently

Use one neutral structure and a few purposeful accents.



```
palette = {  
    'A': '#2B6CB0',  
    'B': '#0F766E',  
    'C': '#D69E2E'  
    ...  
}
```

Define colors once. Reuse them across figures and notebooks.

Small Matplotlib Choices Produce a Large Visual Improvement

alpha

Reveal overlap

`alpha=0.65`

edgecolor

Separate markers

`edgecolor='white'`

zorder

Control drawing order

`zorder=3`

linewidth

Set hierarchy

`linewidth=2.5`

grid

Support reading

`ax.grid(alpha=0.2)`

spines

Remove clutter

```
for s in ['top','right']:  
    ax.spines[s].set_visible(False)
```

Polish should clarify data, not decorate it.

Interactive Python Time

03_python_function_plot.ipynb